Design Rationale

Why the current design needs changing?
The current VLMPipeline is monolithic — it owns the vision encoder, text embeddings, the tokenizer, and the LLM. The InputsEmbedder (which is the de facto "processor") is an internal class hidden in src, not exposed to users. This causes several problems:

- No embedding reuse — you can't get vision embeddings without running the LLM.
- No cache reuse between text generation pipeline and embeddings-only workflows (retrieval, similarity search, etc.). Moving VisionRegistry from Pipeline to Processor allows sharing cached embeddings across multiple pipelines and use cases.
- Tight coupling — `ContinuousBatchingPipeline` duplicates VLM logic via `VLMContinuousBatchingAdapter` instead of sharing a common Processor.
- It is not possible to define target device and plugin config separately for the processor models and the LLM model. (**CVS-162621**)
- Combinatorial overload explosion — `VLMPipeline::generate()` has ~12 overloads for every combination of (string/ChatHistory) × (image/images/videos/none) × (config/properties).
- Not aligned with other libraries, for example HF — HuggingFace's pattern is Processor + Model, which allows users to inspect/modify embeddings between the two steps. HuggingFace alignment HF's canonical VLM flow:

```
processor = AutoProcessor.from_pretrained("model_id")
model = AutoModelForVision2Seq.from_pretrained("model_id")

inputs = processor(images=images, text=prompt, return_tensors="pt")
# → BatchFeature { input_ids, attention_mask, pixel_values }
# OR after internal vision encoding:
# → { inputs_embeds, attention_mask, position_ids }

output_ids = model.generate(**inputs)
text = processor.batch_decode(output_ids)
```

The main flow: the Processor produces a structured input; the model consumes it. The Processor can also be used standalone (e.g., get_image_features()).

Proposed API (C++ Headers)

1. VLMInputs — The structured intermediate

```
// openvino/genai/visual_language/vlm_inputs.hpp

#pragma once
#include <optional>
#include <vector>
```

```cpp
#include "openvino/runtime/tensor.hpp"

namespace ov::genai {

/// @brief Structured inputs for VLM generation, produced by VLMProcessor.
/// Analogous to HuggingFace's BatchFeature returned by a Processor.
struct VLMInputs {
    /// @brief Merged embeddings of text tokens and vision features.
    /// Shape: [1, sequence_length, hidden_size].
    /// The vision encoder outputs have already been projected into
    /// the LLM's embedding space and merged at <image>/<video> positions.
    ov::Tensor inputs_embeds;

    /// @brief Attention mask for the merged sequence.
    /// Shape: [1, sequence_length]. 1 = attend, 0 = ignore.
    ov::Tensor attention_mask;

    /// @brief Position IDs for models that require explicit positioning,
    /// otherwise uninitialized.
    /// (e.g., Qwen2-VL uses 3D MROPE position IDs).
    /// Shape varies by model. May be empty for models using default positions.
    ov::Tensor position_ids;

    /// @brief Optional token type IDs (used by some models like Phi-4-MM),
    /// otherwise uninitialized.
    /// Shape: [1, sequence_length].
    ov::Tensor token_type_ids;
};

} // namespace ov::genai
```

2. VLMProcessor — The public processor class

```cpp
// openvino/genai/visual_language/processor.hpp

#pragma once
#include <filesystem>
#include <string>
#include <vector>
#include "openvino/runtime/tensor.hpp"
#include "openvino/genai/tokenizer.hpp"
#include "openvino/genai/visual_language/vlm_inputs.hpp"
#include "openvino/genai/visual_language/perf_metrics.hpp"
#include "openvino/genai/common_types.hpp"

namespace ov::genai {
```

```cpp
/// @brief A processor for Visual Language Models that handles
/// vision encoding, text tokenization, embedding merging, and
/// chat template application (via internal Tokenizer).
///
/// Analogous to HuggingFace's AutoProcessor - combines an image
/// processor (VisionEncoder) and a tokenizer/embedder into a
/// single preprocessing pipeline.
///
/// Usage:
///   auto processor = VLMProcessor("path/to/models", "GPU");
///   auto inputs = processor.prepare("Describe this image", {image_tensor});
///   auto result = llm.generate(inputs, generation_config);
class OPENVINO_GENAI_EXPORTS VLMProcessor {
public:
    /// @brief Construct a processor from a directory containing
    /// vision encoder, text embeddings model, tokenizer, and configs.
    /// @param models_path Directory with openvino_vision_embeddings_model.xml,
    ///        openvino_text_embeddings_model.xml, tokenizer files, and config.json.
    /// @param device Inference device for vision encoder and embeddings model.
    /// @param properties Device configuration properties.
    VLMProcessor(
        const std::filesystem::path& models_path,
        const std::string& device,
        const ov::AnyMap& properties = {}
    );

    /// @brief Construct from pre-loaded models.
    /// @param models_map Map of model name to (IR string, weights tensor) pairs.
    ///        Expected keys: "vision_embeddings", "text_embeddings",
    ///        and optionally "resampler".
    /// @param tokenizer Pre-initialized tokenizer.
    /// @param config_dir_path Path to directory containing config.json.
    /// @param device_mapping Inference device per model type, e.g.:
    ///        {{"vision_embeddings", "NPU"}, {"text_embeddings", "CPU"}}.
    ///        By default all models run on CPU.
    /// @param properties Per model device configuration properties, e.g.:
    ///        {"PER_MODEL_PROPERTIES":
    ///          {"vision_embeddings", {{"PERFORMANCE_HINT", "LATENCY"}}},
    ///          {"text_embeddings",   {{"NUM_STREAMS", "4"}}}}}.
    ///        Properties from top-level are also applied to all models
    ///        unless overridden in PER_MODEL_PROPERTIES.
    ///        DEVICE_PROPERTIES can be used to specify device-specific properties, e.g.:
    ///        {"DEVICE_PROPERTIES": {"NPU", {{"NPUW_SPECIFIC_PROPERTY", "XXX"}}}}}.
    ///
    /// Usage:
```

```
///   ModelsMap models_map = {
///     {"vision_embeddings", {vision_ir_str, vision_weights_tensor}},
///     {"text_embeddings", {text_ir_str, text_weights_tensor}}
///   };
///   DeviceMapping device_mapping = {
///     {"vision_embeddings", "NPU"},
///     {"text_embeddings", "CPU"}
///   };
///   ov::AnyMap properties = {
///     {"PER_MODEL_PROPERTIES", {
///         {"vision_embeddings", {{"PERFORMANCE_HINT", "LATENCY"}}},
///         {"text_embeddings",   {{"NUM_STREAMS", "4"}}}}
///     }},
//      {"DEVICE_PROPERTIES", {
///         {"NPU", {{"NPUW_SPECIFIC_PROPERTY", "XXX"}}}}}
///     {"GLOBAL_PROPERTY", "value"} // applied to all models unless overridden
///   };
///   auto processor = VLMProcessor(
///       models_map,
///       tokenizer,
///       config_dir_path,
///       device_mapping,
///       properties);
VLMProcessor(
    const ModelsMap& models_map,
    const Tokenizer& tokenizer,
    const std::filesystem::path& config_dir_path,
    const DeviceMapping& device_mapping = {},
    const ov::AnyMap& properties = {}
);

~VLMProcessor();

/// @brief Prepare inputs for VLM generation from a text prompt
/// and optional images/videos.
///
/// Performs the preprocessing pipeline:
/// 1. DOES NOT apply chat template - prompt is treated as already prepared
///    input text, merged with image/video ids.
/// 2. Encodes images/videos through the vision encoder
///    (or reuses from cache using Vision Registry)
/// 3. Tokenizes the text and computes text embeddings.
/// 4. Merges vision features into the text embedding sequence
///    at <image>/<video> placeholder positions.
/// 5. Computes attention mask and position IDs.
///
```

```
/// For using image and video tags in prompt, see:
/// https://openvinotoolkit.github.io/openvino.genai/docs/use-cases/image-processing/
///
/// @param prompt Text prompt, prepared manually by external dependency.
/// @param images RGB image tensors with [NHWC] or [HWC] layout.
/// @param videos Video frame tensors with [NHWC] layout.
/// @return VLMInputs ready to pass to VLMPipeline::generate()
///         or ContinuousBatchingPipeline::add_request().
VLMInputs embed(
    const std::string& prompt,
    const std::vector<ov::Tensor>& images = {},
    const std::vector<ov::Tensor>& videos = {}
);

/// @brief Overload of embed() that accepts a ChatHistory instead of raw string.
/// This calls get_tokenizer().apply_chat_template() internally, so the history
/// is formatted according to the current chat template settings.
///
/// @param history ChatHistory containing system, user, and assistant turns.
/// @param images Optional image tensors.
/// @param videos Optional video tensors.
/// @return VLMInputs with the chat template applied to the history
///         and vision features merged in.
VLMInputs embed(
    const ChatHistory& history,
    const std::vector<ov::Tensor>& images = {},
    const std::vector<ov::Tensor>& videos = {}
);

/// @brief Extract vision embeddings without merging with text.
/// Useful for embedding-only workflows (retrieval, caching, analysis)
/// without invoking the LLM.
///
/// Analogous to HuggingFace's model.get_image_features().
///
/// @param images_or_videos Interpreted as RGB (N,H,W,C) images if N=1 or videos if N>1
/// @return Vector of embedding tensors, one per image or video.
std::vector<ov::Tensor> get_vision_embeddings(
    const std::vector<ov::Tensor>& images_or_videos
);

/// @brief Get the underlying tokenizer.
Tokenizer get_tokenizer() const;

/// @brief Get the vision registry.
/// For advanced users who want to manage vision embeddings and caching directly,
```

```cpp
    /// but also for VLMPipeline internal use, since the registry is now owned
    /// by the processor instead of the pipeline.
    std::shared_ptr<VisionRegistry> get_vision_registry() const;

private:
    class Impl;
    std::unique_ptr<Impl> m_impl;
};

} // namespace ov::genai
```

3. Redesigned VLMPipeline — LLM-only

```cpp
// openvino/genai/visual_language/pipeline.hpp (redesigned)

#pragma once
#include <filesystem>
#include <string>
#include "openvino/genai/generation_config.hpp"
#include "openvino/genai/streamer_base.hpp"
#include "openvino/genai/tokenizer.hpp"
#include "openvino/genai/visual_language/vlm_inputs.hpp"
#include "openvino/genai/visual_language/perf_metrics.hpp"
#include "openvino/genai/common_types.hpp"

namespace ov::genai {

/// @brief A Visual Language Model pipeline that performs text generation
/// given pre-processed VLMInputs from a VLMProcessor.
///
/// This class owns only the LLM (language model). Vision encoding and
/// embedding preparation are handled by VLMProcessor.
///
/// Usage:
///   auto processor = VLMProcessor("path/to/models", "GPU");
///   auto llm = VLMPipeline("path/to/llm_model", processor, "GPU");
///   auto inputs = processor.prepare("Describe this image", {image_tensor});
///   auto result = llm.generate(inputs, generation_config);
class OPENVINO_GENAI_EXPORTS VLMPipeline {
public:
    /// @brief Construct the LLM pipeline from a model directory.
    /// @param llm_model_path Path to directory containing the language model IR.
    /// @param processor A VLMProcessor instance, responsible for vision encoding
    ///                  embedding preparation, and tokenization.
    /// @param device Inference device.
    /// @param properties Device configuration properties.
```

```cpp
VLMPipeline(
    const std::filesystem::path& llm_model_path,
    const VLMProcessor& processor,
    const std::string& device,
    const ov::AnyMap& properties = {}
);

/// @brief Construct from a pre-loaded language model.
/// @param model_str Language model IR as string.
/// @param weights_tensor Model weights.
/// @param processor A VLMProcessor instance.
/// @param device Inference device.
/// @param properties Device configuration properties.
VLMPipeline(
    const std::string& model_str,
    const ov::Tensor& weights_tensor,
    const VLMProcessor& processor,
    const std::string& device,
    const ov::AnyMap& properties = {}
);

~VLMPipeline();

/// @brief Generate text from pre-processed VLMInputs.
/// @param inputs Structured inputs from VLMProcessor::embed().
/// @param generation_config Text generation parameters.
/// @param streamer Optional streamer for token-by-token output.
/// @return Generated text(s) with scores and performance metrics.
VLMDecodedResults generate(
    const VLMInputs& inputs,
    const GenerationConfig& generation_config,
    const StreamerVariant& streamer = std::monostate{}
);

/// @brief Generate with config as property map (for Python bindings convenience).
VLMDecodedResults generate(
    const VLMInputs& inputs,
    const ov::AnyMap& config_map
);

/// @brief Variadic property overload.
template <typename... Properties>
util::EnableIfAllStringAny<VLMDecodedResults, Properties...> generate(
    const VLMInputs& inputs,
    Properties&&... properties
) {
```

```cpp
        return generate(inputs, AnyMap{std::forward<Properties>(properties)...});
    }

    /// ---- Legacy convenience overloads (delegate to internal processor) ----
    /// These use VLMProcessor passed in constructor for backward compatibility.
    /// New code should prefer the VLMProcessor + generate(VLMInputs) pattern.
    /// Should be removed in next releases.

    /// @deprecated Internally calls VLMProcessor::embed() + generate(VLMInputs).
    VLMDecodedResults generate(
        const std::string& prompt,
        const std::vector<ov::Tensor>& images,
        const GenerationConfig& generation_config,
        const StreamerVariant& streamer = std::monostate{}
    );

    /// @deprecated Internally calls VLMProcessor::embed() + generate(VLMInputs).
    VLMDecodedResults generate(
        const std::string& prompt,
        const ov::AnyMap& config_map
    );

    /// @brief Get the tokenizer (for output detokenization if needed).
    Tokenizer get_tokenizer() const;

    GenerationConfig get_generation_config() const;
    void set_generation_config(const GenerationConfig& config);

private:
    class Impl;
    std::unique_ptr<Impl> m_impl;
};

} // namespace ov::genai
```

4. ContinuousBatchingPipeline — VLM-aware add_request()

```cpp
// In continuous_batching_pipeline.hpp - additions to existing class

// New add_request overload accepting VLMInputs:

/// @brief Add a VLM request with pre-processed inputs from VLMProcessor.
/// @param request_id Unique request identifier.
/// @param inputs Pre-processed VLMInputs containing merged embeddings.
/// @param sampling_params Generation configuration.
/// @return Handle to monitor and read generation results.
```

```
GenerationHandle add_request(
    uint64_t request_id,
    const VLMInputs& inputs,
    const ov::genai::GenerationConfig& sampling_params
);

// New generate overload for batch VLM:

/// @brief Batch generate from pre-processed VLMInputs.
std::vector<VLMDecodedResults> generate(
    const std::vector<VLMInputs>& inputs,
    const std::vector<GenerationConfig>& sampling_params,
    const StreamerVariant& streamer = std::monostate{}
);
```

Complete usage examples

Example 1: VLMPipeline — basic single image

```
// Separate construction: processor owns vision models, pipeline owns LLM
auto processor = ov::genai::VLMProcessor("path/to/models", "GPU");
auto llm = ov::genai::VLMPipeline("path/to/models", processor, "GPU");

// Embed: encodes images, tokenizes text, merges embeddings
auto inputs = processor.embed(
    "<|im_start|><ov_image_0> Describe this image<|im_end|><|im_start|>", {image_tensor});

// Generate: runs LLM on pre-merged embeddings
ov::genai::GenerationConfig config;
config.max_new_tokens = 256;
auto result = llm.generate(inputs, config);
std::cout << result.texts.at(0) << std::endl;
```

Example 2: Custom device mapping and plugin configuration

```
// Load models from memory (e.g., read from custom storage or simply from disk)
ov::genai::ModelsMap models_map = {
    {"vision_embeddings", {vision_ir_str, vision_weights_tensor}},
    {"text_embeddings", {text_ir_str, text_weights_tensor}},
    {"resampler", {resampler_ir_str, resampler_weights_tensor}}
};

// Assign each model to a different device
ov::genai::DeviceMapping device_mapping = {
    {"vision_embeddings", "NPU"},
    {"text_embeddings", "CPU"},
    {"resampler", "GPU"}
```

```
};

// Properties: per-model, per-device, and global
ov::AnyMap properties = {
    // Per-model properties override global ones for the specified model
    {"PER_MODEL_PROPERTIES", ov::AnyMap{
        {"vision_embeddings", ov::AnyMap{{"PERFORMANCE_HINT", "LATENCY"}}},
        {"text_embeddings",   ov::AnyMap{{"NUM_STREAMS", "4"}}}
    }},
    // Per-device properties apply to all models on that device
    {"DEVICE_PROPERTIES", ov::AnyMap{
        {"NPU", ov::AnyMap{{"NPUW_SPECIFIC_PROPERTY", "XXX"}}},
        {"GPU", ov::AnyMap{{"GPU_QUEUE_PRIORITY", "HIGH"}}}
    }},
    // Global property - applied to all models unless overridden above
    {"CACHE_DIR", "/tmp/ov_cache"}
};

ov::genai::Tokenizer tokenizer("path/to/tokenizer");

auto processor = ov::genai::VLMProcessor(
    models_map,
    tokenizer,
    "path/to/config_dir",  // contains image/video preprocessing settings
    device_mapping,
    properties
);

// LLM runs on GPU with its own properties if needed
auto llm = ov::genai::VLMPipeline("path/to/models", processor, "GPU");

auto inputs = processor.embed(
    "<|im_start|><ov_image_0> Describe this image<|im_end|><|im_start|>", {image_tensor});
auto result = llm.generate(inputs, config);
```

Example 3: Multi-turn chat with multiple images via ChatHistory

```
auto processor = ov::genai::VLMProcessor("path/to/models", "GPU");
auto llm = ov::genai::VLMPipeline("path/to/models", processor, "GPU");

// Build a ChatHistory with multipart content (OpenAI-like format)
ov::genai::ChatHistory history = {
    {{"role", "system"}, {"content", "You are a helpful assistant."}},
    {{"role", "user"}, {"content", {
        {{"type", "text"}, {"text", "Compare these two images:"}},
        {{"type", "image"}},
```

```
        {{"type", "image"}}
    }}}
};

// embed() applies the chat template and merges vision features
auto inputs = processor.embed(history, {image1, image2});
auto result = llm.generate(inputs, config);
```

Example 4: Embedding extraction (no LLM)

```
auto processor = ov::genai::VLMProcessor("path/to/models", "GPU");

// Extract vision embeddings only - no LLM needed
auto embeddings = processor.get_vision_embeddings({image1, image2});
// embeddings[0].get_shape() → {num_patches, hidden_size}

// Use for retrieval, similarity, caching, etc.
```

Example 5: ContinuousBatchingPipeline — concurrent VLM requests

```
auto processor = ov::genai::VLMProcessor("path/to/models", "GPU");
auto llm = ov::genai::ContinuousBatchingPipeline("path/to/models", scheduler_config, process

// Embed multiple requests
auto inputs1 = processor.embed(
    "<|im_start|><ov_image_0> What is in this photo?<|im_end|><|im_start|>", {photo1});
auto inputs2 = processor.embed(
    "<|im_start|><ov_image_0> Describe this diagram<|im_end|><|im_start|>", {diagram});

// Submit to continuous batching
auto handle1 = llm.add_request(1, inputs1, config);
auto handle2 = llm.add_request(2, inputs2, config);

// Process until done
while (llm.has_non_finished_requests()) {
    llm.step();
}

auto results1 = handle1->read_all();
auto results2 = handle2->read_all();
```

Example 6: Inspecting / modifying embeddings between processor and LLM

```
auto processor = ov::genai::VLMProcessor("path/to/models", "GPU");
auto llm = ov::genai::VLMPipeline("path/to/models", processor, "GPU");
```

```
auto inputs = processor.embed(
    "<|im_start|><ov_image_0> Describe this image<|im_end|><|im_start|>", {image_tensor});

// User can inspect or transform embeddings before generation
std::cout << "Embedding sequence length: "
          << inputs.inputs_embeds.get_shape()[1] << std::endl;

// For example, truncate if too long
// ... modify inputs.inputs_embeds, inputs.attention_mask ...

auto result = llm.generate(inputs, config);
```